

UD 6 y 7. UTILIZACIÓN DE OBJETOS Y DESARROLLO DE CLASES

**Programación – 1º DAM
Curso 2025/2026**

Introducción

La programación orientada a objetos es un paradigma de programación totalmente diferente al método clásico de programación, el cual utiliza objetos y su comportamiento para resolver problemas y generar programas y aplicaciones informáticas.

Con la POO aumenta la modularidad de los programas y la reutilización de los mismos.

Además, se diferencia de la programación clásica porque utiliza técnicas nuevas como el polimorfismo, el encapsulamiento, la herencia, etc.

Introducción al concepto de objeto

Un objeto en Java no es ni más ni menos que lo mismo que un objeto en el mundo real. No tiene existencia física, pero sí existe como "algo" que contiene información sobre sí mismo, sobre su estado, y con el que puedes interactuar. Los programas realizados mediante el paradigma de la POO solamente tienen objetos.

Todos los objetos pertenecen a una clase.

Ejemplo:

Todos los objetos de la clase pájaro se identificarán, entre otros atributos, con un nombre, un color de plumaje, una edad y si son domésticos o no.

Por ejemplo, mi loro Felipe pertenecería a la clase pájaro.

Clases

La POO (Programación Orientada a Objetos) se inspira en una abstracción del mundo real, en la que todos los objetos se clasifican en grupos.

Por ejemplo, todos los mamíferos de cuatro patas que dicen “guau” se engloban dentro del grupo de los perros.

O si observamos a Paco, Pepa y Miguel, vemos que los tres pertenecen al mismo grupo, son personas.

En la POO, a cada uno de estos grupos se les denomina **clase**.

Clases

Las clases son los moldes de los cuales se generan los objetos.

Los objetos se instancian y se generan → **Instancia=Objeto**.

En un símil con la costura, las clases son los patrones y los objetos son las prendas.

Cuando se escribe un programa o aplicación OO, lo que se hace es definir las clases de objetos dotándolas de estado y comportamiento y cuando se ejecute el programa se crearán los objetos.

Cuando se programa, las clases se escriben en ficheros ASCII con el mismo nombre que la clase y extensión .java.

Clases

Podemos definir cada grupo o clase mediante las **propiedades** y **comportamientos** que presentan todos sus miembros. Una **propiedad** es un dato que conocemos de cada miembro del grupo, mientras que un **comportamiento** es algo que puede hacer.

Vamos a definir la clase mediante estos dos elementos:

- Propiedades: un nombre, una edad y una estatura, Tanto Pepa, como Paco, como Miguel tienen un nombre, una edad y una estatura; son datos que en cada uno tendrá un valor distinto.
- Comportamiento: Pepa, Paco y Miguel (y todas las personas) pueden saludar, crecer o cumplir años, por ejemplo.

Clases

Como vemos, es posible definir una clase mediante un conjunto de propiedades y comportamientos. La sintaxis para definir una clase usa la palabra reservada **class**:

```
[modificadores] class nombre_clase ([parámetros]) {  
    [Atributos]  
    [Métodos]  
}
```

En [] se han etiquetado los elementos opcionales de la clase.

Clases

Es muy común ver definiciones de clases como:

```
public class Nombre_clase
```

La palabra reservada `public` indica que la clase puede ser accedida por cualquier clase que necesite de su utilización.

Los atributos dentro de una clase pueden ser desde cero a muchos.

Una clase puede tener cero o muchos métodos y se corresponden con los procedimientos o funciones de otros lenguajes de programación.

Clases

A continuación, se enumeran características importantes de las clases:

1. Todas las variables y funciones de Java deben pertenecer a una clase. No hay variables y funciones globales.
2. Si una clase deriva de otra (extends), hereda todas sus variables y métodos.
3. Java tiene una jerarquía de clases estándar de la que pueden derivar las clases que crean los usuarios.
4. Una clase sólo puede heredar de una única clase (en Java no hay herencia múltiple). Si al definir una clase no se especifica de qué clase deriva, por defecto la clase deriva de Object. La clase Object es la base de toda la jerarquía de clases de Java.

Clases

5. En un fichero se pueden definir varias clases, pero en un fichero no puede haber más que una clase public. Este fichero se debe llamar como la clase public que contiene con extensión *.java. Con algunas excepciones, lo habitual es escribir una sola clase por fichero.
6. Si una clase contenida en un fichero no es public, no es necesario que el fichero se llame como la clase.
7. Los métodos de una clase pueden referirse de modo global al objeto de esa clase al que se aplican por medio de la referencia this.
8. Las clases se pueden agrupar en packages, introduciendo una línea al comienzo del fichero (package packageName;).

Atributos

Los datos que definen una clase se denominan **atributos**. Por ejemplo, la clase Vehículo puede definirse mediante los atributos matricula, color, marca y modelo. O los atributos vistos de la clase Persona, nombre, edad y estatura.

La forma de declarar los atributos en una clase es:

```
[modificadores] class Nombre_clase ([parámetros]) {  
    tipo atributo1;  
    tipo atributo2;  
    ...  
}
```

Atributos

El tipo especificado en tipo puede ser cualquier tipo primitivo, o una clase, como veremos después.

Ejemplo: el código para definir nuestra Persona será:

```
class Persona {  
    String nombre;  
    int edad;  
    double estatura;  
}
```

Inicialización

Se puede asignar un valor por defecto a los atributos de una clase; esto se realiza en la propia declaración de la forma.

Ejemplo: el código para definir nuestra Persona será:

```
class NombreClase {  
    tipo atributo1=valor;  
    ...  
}
```

Inicialización

En general, los atributos pueden cambiar durante la ejecución de un programa, salvo que sean declarados con el modificador **final**. En este caso, el atributo será una constante que, una vez inicializado, no podrá cambiar de valor.

Si suponemos que el DNI de una persona no cambia una vez asignando:

```
class Persona {  
    String nombre;  
    int edad;  
    double estatura;  
    final String dni; //una vez asignado no podrá cambiarse  
}
```

Objetos

Un objeto tiene una serie de características:

- Identidad: cada objeto es único y diferente de otro objeto. Ejemplo: Mi loro Felipe es diferente a otros loros.
- Estado: el estado serán los valores de los atributos del objeto, en el caso de los objetos de clase pájaro serían nombre, color, edad, doméstico, etc.
- Comportamiento: el comportamiento serían los métodos o procedimientos que realiza dicho objeto. Dependiendo de tipo o clase de objeto, éstos realizarán unas operaciones u otras (volar, cantar, hablar, etc)

Objetos

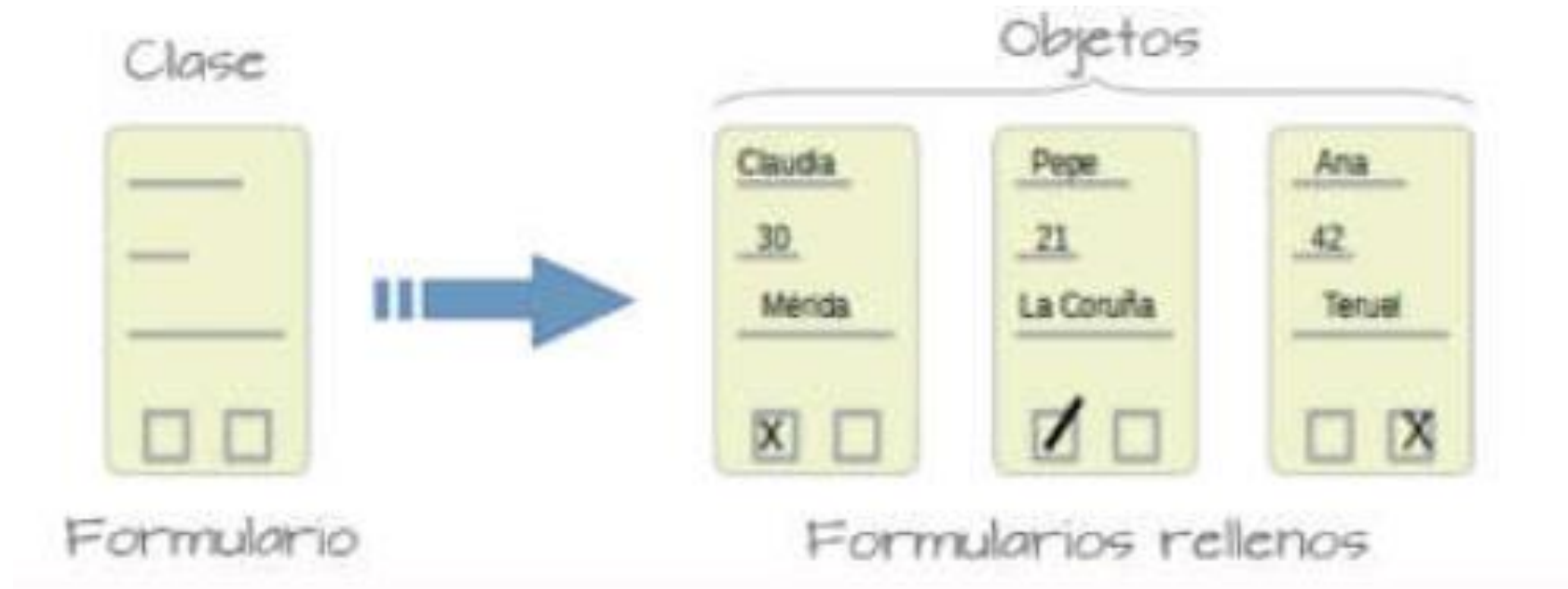
Los elementos que pertenecen a una clase se denominan instancias u objetos.

Cada uno tiene sus propios valores de los atributos definidos en la clase.

Por **ejemplo**, supongamos una clase es un formulario donde se solicitan una serie de datos. Cada formulario relleno recoge distintos valores para los datos que se solicitan, siendo cada uno de los formularios rellenos, en nuestro símil, un objeto concreto.

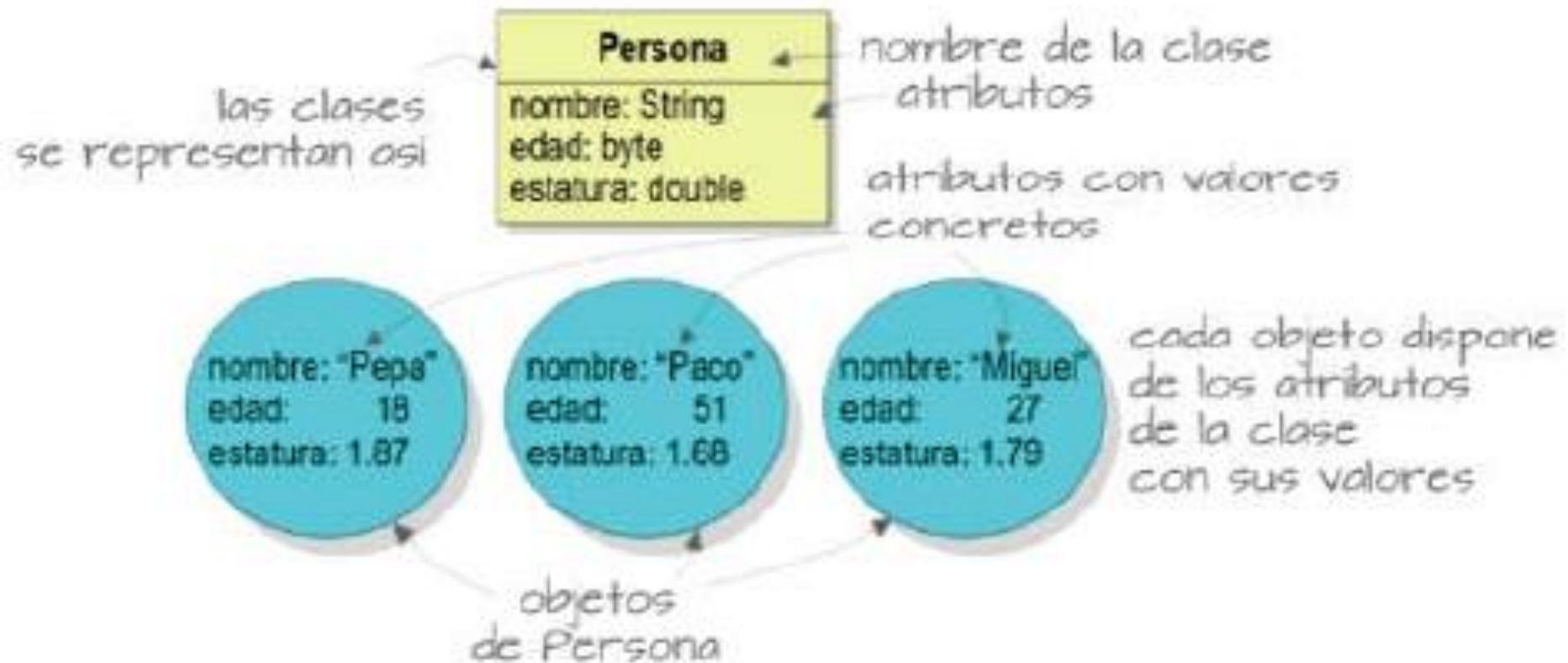
Todos los formularios cumplimentados (los objetos) tendrán la misma estructura (han sido creados con la misma clase). Sin embargo, cada formulario relleno (objeto) tendrá distintos valores: distintos nombres, direcciones, etc.

Objetos



Objetos

En nuestro ejemplo Persona, Pepa, Paco y Miguel, cada uno de ellos es un objeto de la clase Persona.



Objetos. Variables referencia

Antes de construir un objeto necesitamos declarar una variable cuyo tipo sea su clase. La declaración sigue las mismas reglas que las variables de tipo primitivo.

```
Clase nombreVariable;
```

donde `Clase` será el nombre de cualquier clase disponible.

Ejemplo:

```
Persona p; //p es una variable de tipo Persona
```

La diferencia entre una variable de tipo primitivo y una variable de tipo referencia, es que mientras una variable de tipo primitivo almacena directamente un valor, una variable de tipo clase almacena la referencia de un objeto.

Objetos. Operador new

La forma de crear objetos es mediante el operador **new**.

```
p=new Persona();
```

En este caso, crear un objeto de tipo Persona y asigna su referencia a la variable p.

El operador new, primero busca en memoria un hueco disponible donde construir el objeto. Este, dependiendo de su tamaño, ocupará cierto número consecutivo de bloques de memoria. Por último, devuelve la referencia del objeto recién creado, que se asigna a la variable p.

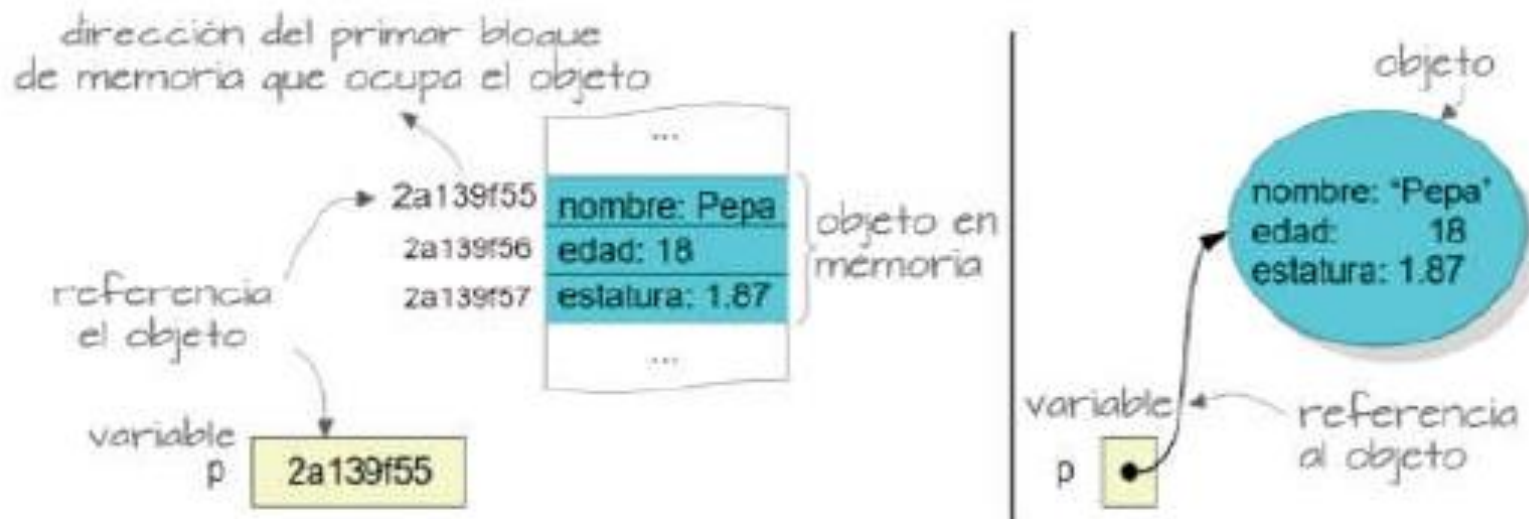
Objetos. Operador new

Podemos comprobar qué aspecto tiene una referencia ejecutando:

```
p=new Persona();
```

```
System.out.println(p); //muestra en consola la  
referencia del objeto
```

A la hora de trabajar con referencias, es bastante más sencillo pensar en ellas como flechas que se dirigen desde la variable hacia el objeto:



Objetos. Referencia null

El valor literal **null** es una referencia nula. Dicho de otra forma, una referencia a ningún bloque de memoria. Cuando declaramos una variable referencia se inicializa por defecto a null.

Si intentamos acceder a una variable de referencia nula, producirá un error que terminará la ejecución del programa de forma inesperada.

Ejemplo:

```
Persona p;      //se inicializa por defecto a null  
p.nombre;      //¡error!
```

Esta última instrucción genera un error del tipo: Null pointer exception, que significa que estamos intentando acceder a los atributos de un objeto que no existe.

Objetos. Referencia null

El literal **null** se puede asignar a cualquier variable referencia.

```
Persona p;      //p referencia un objeto
```

```
p = null;       //p no referencia a nada
```

Objetos. Recolector de basura

Existen tres formas de conseguir que un objeto no esté referenciado:

- Es posible, aunque no tenga mucho sentido, crear un objeto y no asignarlo a ninguna variable.

```
new Persona();
```

- Otra posibilidad es asignar **null** a todas las variables que contenían una referencia a un objeto.
- También podemos asignar un objeto distinto a la variable.

```
Persona p=new Persona(); //objeto 1
```

```
p = new Persona(); //objeto 2. Ahora el objeto 1 queda  
sin referencia
```


Objetos. Recolector de basura

En todos los casos, el objeto se queda perdido en memoria, es decir, no existe forma de acceder a él. Sin embargo, está ocupando memoria.

Si este comportamiento se repite demasiado, es posible que se agote toda la memoria disponible, lo que impediría el normal funcionamiento del ordenador.

Para evitar este problema, Java dispone de un mecanismo llamado **recolector de basura**, *garbage collector*, que se ejecuta de vez en cuando de forma transparente al usuario, y se encarga de comprobar, uno a uno, todos los objetos de la memoria. Si alguno de ellos no estuviera referenciado por ninguna variable, se destruye, liberando la memoria que ocupa.

Métodos

Hemos declarado clases con atributos, pero también disponen de comportamientos. En POO a estos comportamientos u operaciones que pueden realizar los objetos de una clase se les denomina **métodos**.

Los métodos no son más que funciones que se implementan dentro de una clase. Su sintaxis es:

```
public class NombreClase{  
    ...      //declaración de atributos  
    tipo nombreMétodo (parámetros){  
        cuerpo del método  
    }  
}
```

Métodos

La definición de un método es la de una función, sustituyendo *cuerpo del método* por un bloque de instrucciones.

Hasta ahora todas las funciones (métodos de la clase Main) que hemos implementado han sido estáticos por razones que se explicarán después.

Sin embargo, en general, todos los métodos de una clase no tienen por qué ser estáticos.

Métodos

Ejemplo:

```
public class Persona{  
    String nombre;  
    int edad;  
    double estatura;  
    void saludar(){  
        System.out.println("Hola soy "+nombre);  
        System.out.println("Encantado de conocerte");  
    }  
    void cumplirAños(){  
        edad++; //incrementamos la edad en 1  
    }  
    void crecer(double incremento){  
        estatura+=incremento; //la estatura aumenta cierto incremento  
    }  
}
```



Métodos

A partir de ahora, los objetos de tipo Persona pueden invocar sus métodos utilizando un punto (.), al igual que se hace con los atributos.

Ejemplo:

```
Persona p;
```

```
P=new Persona();
```

```
p.edad=18;
```

```
p.cumplirAnios();           //¡Felicidades!           La           edad           se  
incrementa
```

```
System.out.println(p.edad); //mostrará 19
```

Métodos

Los métodos de una clase tienen acceso a las siguientes variables: variables locales declaradas dentro del método, parámetros de entrada y atributos de la clase. Asimismo, tiene acceso a los demás métodos de la clase.

Métodos. Ámbito de las variables y atributos

El ámbito de una variable define en qué lugar puede usarse y coincide con el bloque en el que se declara la variable, que puede ser:

- **Bloque de una estructura de control:** if, if-else, switch, while, do-while o for. Las variables declaradas en este ámbito se llaman **variables de bloque**.
- Una **función** o **método**: la variables declaradas aquí se denominan **variables locales**.

Con la POO, aparece un nuevo ámbito:

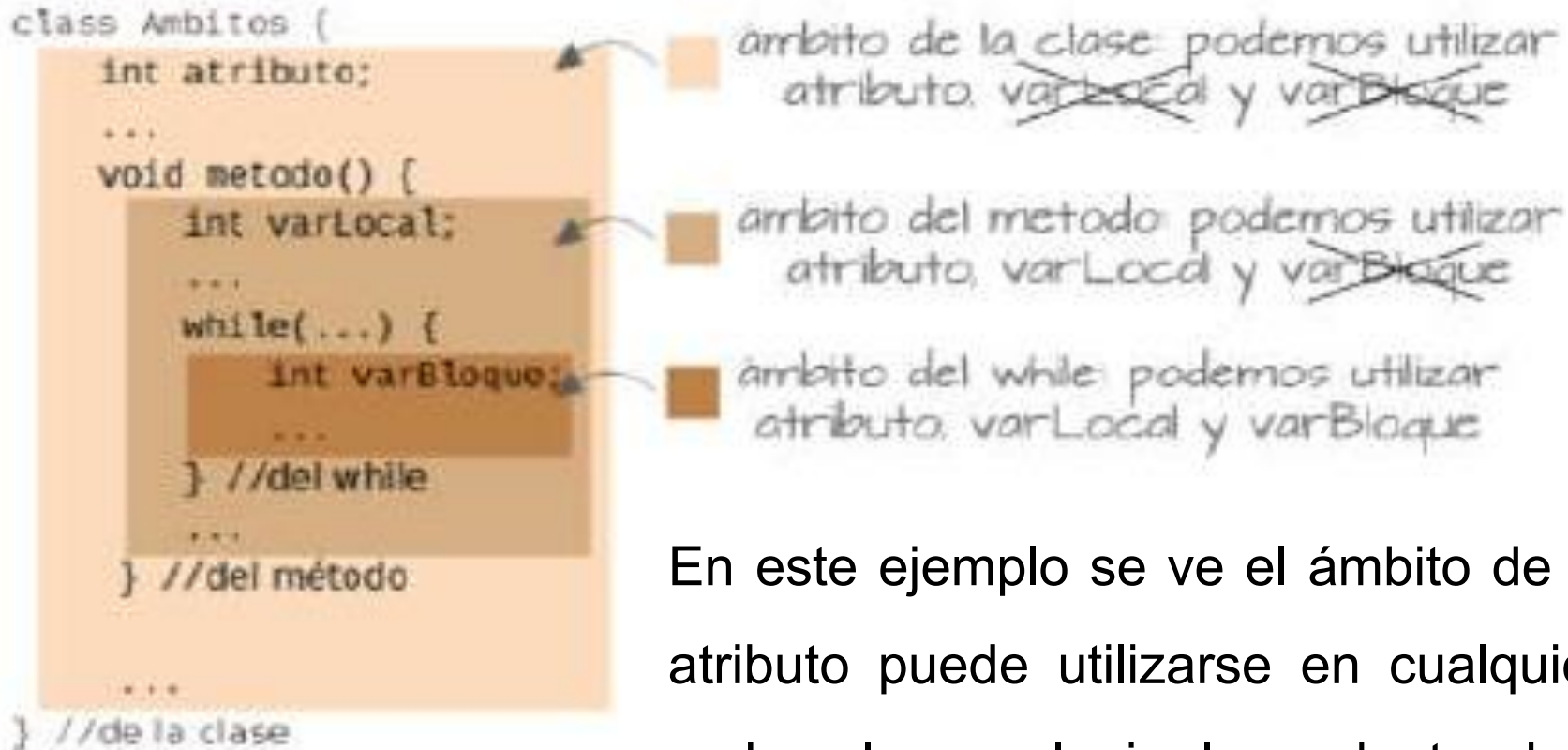
- La **clase**: cualquier miembro (atributo o método) definido en una clase podrá ser utilizado en cualquier lugar de ella. Consideramos que los atributos son **variables de la clase**.

Métodos. Ámbito de las variables y atributos

Un ámbito puede contener a otros ámbitos, formando una estructura jerárquica. Por ejemplo, una clase puede contener dos métodos, y estos, distintos bloques de, por ejemplo, una estructura while o if.

Una variable puede utilizarse en el ámbito o bloque en el que se declara, que incluye sus bloques internos. Sin embargo, no ocurre lo contrario: una variable no podrá utilizarse en el ámbito padre del bloque en el que se declara. Por ejemplo, un atributo puede emplearse dentro de un método y, una variable local dentro del bloque de una estructura de control de un método. Pero, en cambio, no podremos usar una variable local fuera de su método, ni una variable del bloque fuera de él.

Métodos. Ámbito de las variables y atributos



En este ejemplo se ve el ámbito de tres variables, donde atributo puede utilizarse en cualquier lugar de la clase; varLocal en cualquier lugar dentro del método en el que se declara; por último, varBloque puede usarse solo dentro del bloque de instrucciones de la estructura while.

Métodos. Ocultación de atributos

Dos variables declaradas en ámbitos anidados no pueden tener el mismo identificador, ya que esto genera un error. Sin embargo, existe una excepción cuando una variable local en un método tiene el mismo identificador que un atributo de la clase. En este caso, dentro del método, la variable local tiene prioridad sobre el atributo, es decir, que al utilizar el identificador se accede a la variable local y no al atributo. **Ejemplo:**

```
public class Atributo{  
    int edad;    //atributo entero  
    void método{  
        double edad;    //variable local. Oculta al atributo edad  
        edad=8.2;  
    }  
}
```

Métodos. Objeto this

La palabra reservada **this** permite utilizar un atributo incluso cuando ha sido ocultado por una variable local. De igual manera que cada uno se refiere a sí mismo como yo, aunque tengamos un nombre que los demás utilizan para identificarnos, las clases se refieren a sí mismas como this, que es una referencia al objeto actual.

Al escribir this en el ámbito de una clase se interpreta como la propia clase y permite acceder a los atributos aunque se encuentren ocultos. Ejemplo:

```
public class Ambito{  
    int edad;    //atributo entero  
    void método{  
        double edad;    //oculta el atributo edad (que es entero)  
        edad = 20.0;    //variable local, no el atributo  
        this.edad = 30;  
    }  
}
```

Atributos y métodos estáticos

Un **atributo estático**, también llamado **atributo de la clase**, es aquel del que no existe una copia en cada objeto. Todos los objetos de una misma clase comparten su valor en todas las instancias.

Características:

- Se declara mediante la palabra reservada **static**.
- Solo existe **una copia** en memoria.
- Se inicializa en el momento de cargar la clase en memoria.
- Se accede usando el nombre de la clase.
- Es útil para valores compartidos (contadores, constantes, configuraciones).

Atributos y métodos estáticos

Ejemplo:

```
class Persona {
    static String especie = "Humano"; // atributo estático
    String nombre;
    Persona(String nombre) {
        this.nombre = nombre;
    }
}

public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona("Ana");
        Persona p2 = new Persona("Luis");

        System.out.println(Persona.especie); // acceso recomendado
        System.out.println(p1.especie); // funciona, pero no es buena
        práctica
    }
}
```

Atributos y métodos estáticos

También podemos declarar **métodos estáticos**. Son aquellos que **pueden ejecutarse sin crear un objeto** y, por tanto, no pueden utilizar ningún atributo que no sea estático.

La forma de invocar un método estático es, igual que con los atributos estáticos, mediante el nombre de la clase.

Desde un método estático solo se puede invocar directamente métodos y atributos estáticos. (Por esta razón, hasta ahora todos nuestros métodos eran estáticos, porque eran invocados desde el main que, a su vez, es un método estático).

No obstante, dentro de un método estático se pueden crear objetos de cualquier clase, y desde él invocar miembros no estáticos definidos en ella.

Atributos y métodos estáticos

Características:

- Se llaman desde la clase.
- Solo pueden acceder directamente a **miembros estáticos**.
- No pueden usar this ni super.

Atributos y métodos estáticos

Ejemplo:

```
class Matematica {  
  
    static int sumar(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        int resultado = Matematica.sumar(5, 3);  
        System.out.println(resultado);  
    }  
}
```


Atributos y métodos estáticos

```
public class Pajaro {  
    private static int numPajaros=0;  
    private char color;  
    private int edad;  
  
    public Pajaro() {  
        color='v';  
        edad=0;  
        nuevoPajaro();  
    }  
  
    public Pajaro(char c, int e) {  
        color=c;  
        edad=e;  
        nuevoPajaro();  
    }  
  
    static void nuevoPajaro() {  
        numPajaros++;  
    }  
  
    static void muestraPajaros() {  
        System.out.println(numPajaros);  
    }  
  
    public static void main (String[] args) {  
        Pajaro p1, p2;  
        p1=new Pajaro();  
        p2=new Pajaro('a', 3);  
  
        Pajaro.muestraPajaros();  
    }  
}
```

Constructores

Por defecto, cuando creamos un objeto, los atributos a los que no se les asigna un valor en su declaración se inicializan, dependiendo de su tipo, de la siguiente manera: cero para valores numéricos y char, null para referencias y false para booleanos.

Si embargo, generalmente, antes de utilizar un objeto podemos asignar determinados valores a cada uno de sus atributos. Por ejemplo, si queremos crear un objeto de tipo Persona con nombre “Claudia”, edad de 8 años y estatura de 1’20m. Haremos:

```
Persona p = new Persona(); //creamos el objeto
p.nombre = "Claudia"; //asignamos valores
p.edad = 8;
p.estatura = 1.20;
```

Constructores

Cada vez que creemos un objeto será necesario hacerlo si no queremos trabajar con los valores por defecto. El operador **new** facilita esta tarea mediante los constructores.

Un **constructor** es un método especial que debe tener el mismo nombre que la clase, sin tipo devuelto ni void, y se ejecuta inmediatamente después de crear el objeto.

El **objetivo principal** de un constructor es **asignar valores a los atributos**.

Al constructor, como a cualquier otro método, se le pueden pasar parámetros y se puede sobrecargar.

Constructores

Cada vez que creemos un objeto será necesario hacerlo si no queremos trabajar con los valores por defecto. El operador **new** facilita esta tarea mediante los constructores.

Un **constructor** es un método especial que debe tener el mismo nombre que la clase, sin tipo devuelto ni void, y se ejecuta inmediatamente después de crear el objeto.

El **objetivo principal** de un constructor es **asignar valores a los atributos**.

Al constructor, como a cualquier otro método, se le pueden pasar parámetros y se puede sobrecargar.

Constructores

Ejemplo:

```
class Persona {  
    ...  
    Persona (String nombre, int edad, double estatura){  
        this.nombre=nombre; //asigna el parámetro al atributo  
        this.edad=edad;  
        this.estatura=estatura;  
    }  
    . . .  
}
```

Constructores

La llamada al constructor con los valores de los parámetros de entrada se hace por medio del operador new.

Para el ejemplo anterior sería:

```
Persona p = new Persona("Claudia", 6, 1.20);
```

```
//creamos el objeto y lo inicializamos mediante el  
constructor
```

Constructores

Los atributos declarados como **final** también se pueden inicializar pasando sus valores como parámetros al constructor, no es necesario hacerlo en el sitio donde se declaran.

Por defecto, aunque no lo indiquemos, toda clase contiene un constructor, que comúnmente se le llama **constructor por defecto**. Este constructor no recibe ningún parámetro y, por tanto, todos los atributos de la clase tendrán como valor el valor por defecto que le corresponda según su tipo de datos.

Constructores. Constructor por defecto

Cuando en una clase no se implementa ningún constructor, Java se encarga de crear uno que se denomina **constructor por defecto**. Este no usa parámetros de entrada e inicializa los atributos a cero, false o null según el tipo, si no están ya inicializados en su declaración.

En cuanto se implementa un constructor en una clase, el constructor por defecto deja de estar disponible.

Constructores. Sobrecarga

Ejemplo constructor por defecto:

```
class Persona {  
    String nombre;  
    int edad;  
    double estatura;  
    public Persona() {}  
}  
  
/*tras crear un objeto Persona con este constructor, los  
valores que tomará será:*/  
    nombre=null;  
    edad=0;  
    estatura=0.0
```

Si solo necesitamos el constructor por defecto, no es necesario indicarlo.

Constructores. Sobrecarga

La **sobrecarga** de métodos o de constructores es una técnica en la que la clase puede tener un número de constructores o métodos con distinto número de parámetros.

El compilador diferencia estos constructores teniendo en cuenta el número de parámetros y el tipo de datos de cada uno de ellos.

```
class Persona {  
    String nombre;  
    int edad;  
    double estatura;  
    final String dni; //una vez asignado no podrá cambiarse  
}
```

Constructores. Sobrecarga

Como decíamos antes, cuando existe sobrecarga de constructores, si se quiere usar el constructor por defecto, es obligatorio especificarlo.

Ejemplo:

```
public Persona (String nombre, int edad, double estatura){  
    this.nombre=nombre;  
    this.edad=edad;  
    this.estatura=estatura;  
}  
  
public Persona(String nombre){  
    this.nombre=nombre;  
}  
  
//si se quiere usar el constructor por defecto, hay que ponerlo  
public Persona() {}
```

Constructores. Sobrecarga

Como hemos dicho, la sobrecarga de constructores en Java permite múltiples constructores en una clase, cada uno con diferentes listas de parámetros.

Algunas de sus ventajas son:

- **Flexibilidad en la creación de objetos:** permite inicializar objetos de varias maneras, dependiendo del número o tipo de parámetros.
- **Reutilización de código:** puede reutilizar la lógica del constructor llamando a un constructor desde otro, usando la palabra clave `this()` (herencia).
- **Legibilidad mejorada:** ayudan a que el código sea más intuitivo al ofrecer opciones de constructor específicas para diferentes necesidades de inicialización.

Constructores. Sobrecarga

Ejemplo:

```
class Persona {  
    ...  
    //constructor que asigna valores a todos los atributos  
    Persona (String nombre, int edad, double estatura) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.estatura = estatura;  
    }  
  
    //constructor sobrecargado: solo asigna el nombre  
    Persona (String nombre) {  
        this.nombre = nombre;  
        estatura = 1.0; //valor arbitrario para la estatura  
        //al no asignar la edad se inicializa por defecto: a 0  
    }  
}  
  
Persona a = new Persona("Pepe", 20, 1.90);  
  
Persona b = new Persona("Dolores");
```

Constructores. **this()**

Cuando una clase dispone de un conjunto de constructores sobrecargados, es posible que un constructor invoque a otro y así reutilice su funcionalidad. Para eso se usa el constructor genérico **this()**, en lugar del constructor por su nombre. La forma de distinguir los distintos constructores, igual que en cualquier método sobrecargado, es mediante el número y el tipo de los parámetros de entrada.

Constructores. this()

Vamos a redefinir el constructor de Persona al que solo se le pasa el nombre usando **this()**:

```
class Persona {  
    ...  
    //constructor que asigna valores a todos los atributos  
    Persona (String nombre, int edad, double estatura) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.estatura = estatura;  
    }  
  
    //constructor sobrecargado que solo asigna el nombre  
    Persona (String nombre) {  
        this(nombre, 0, 1.0); //invoca al primer constructor  
        //la edad se pone a 0 y la estatura a 1.0  
    }  
}
```

Importante: en caso de usar this(), **tiene que ser siempre la primera instrucción de un constructor**; en otro caso se producirá un error.

Modificadores de acceso

Una clase será visible por otra, o no, dependiendo de si se ubican en el mismo paquete y de los modificadores de acceso que utilice. Estos modifican su visibilidad, permitiendo que se muestre u oculte.

De igual manera que podemos modificar la visibilidad entre clases, es posible modificar la visibilidad entre los miembros de distintas clases, es decir, qué atributos y métodos son visibles para otras clases.

Modificadores de acceso para clases

Debido a la estructura de las clases, organizadas en paquetes, que utiliza Java, dos clases cualesquiera pueden definirse de las siguientes formas:

- Clases vecinas: cuando ambas pertenecen al mismo paquete.
- Clases externas: cuando se han definido en paquetes distintos.

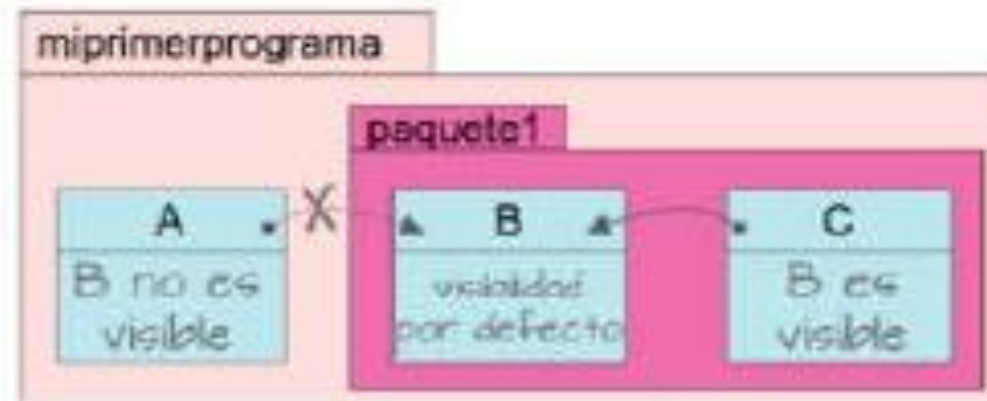


Modificadores de acceso para clases

Visibilidad por defecto

Cuando definimos una clase sin utilizar ningún modificador de acceso se dice que se usa **visibilidad por defecto**, que hace que solo sea visible por sus clases vecinas.

```
package miprimerprograma.paquete1;  
class B { //sin modificador de acceso  
    ...  
}
```



Modificadores de acceso para clases

Visibilidad total

Si queremos que una clase, además de ser visible por sus clases vecinas, también lo sea para todas las clases externas, tenemos que usar el modificador ***public***. Es decir, el modificador public proporciona visibilidad total a la clase.

Modificadores de acceso para clases

Visibilidad total

Redefinimos B para que tenga visibilidad total:

```
package miprimerprograma.paquetel;  
public class B { //clase marcada como pública  
    ...  
}
```

A partir de ahora, cualquier clase, vecina o externa, puede crear objetos o acceder a los miembros públicos de B. Lo único que necesita una clase externa, como A, para acceder a B es importarla.

```
package miprimerprograma;  
import miprimerprograma.paquetel.B; //ahora A puede usar la clase B  
  
class A {  
    ...  
}
```

Modificadores de acceso para clases

Visibilidad total

Se importan las clases, no los paquetes. Si queremos importar todas las clases públicas de un paquete en una sola sentencia de importación, se usa el asterisco.

```
package miprimerprograma;
import miprimerprograma.paquetel.*; //A puede usar cualquier clase pública
                                     //del paquete miprimerprograma.paquetel

class A {
    ...
}
```

Modificadores de acceso para clases

	Visible desde...	
	clases vecinas	clases externas
sin modificador	✓	
public	✓	✓

Modificadores de acceso para miembros

También podemos modificar la visibilidad de sus miembros (atributos y métodos).

Que un atributo sea visible significa que podemos acceder a él, tanto para leer como para modificarlo.

Que un método sea visible significa que puede ser invocado.

Para que un miembro sea visible, es indispensable que se clase también lo sea. Es evidente que, si no podemos acceder a una clase, no existe forma alguna de acceder a sus miembros.

Modificadores de acceso para miembros

Importante: cualquier miembro es siempre visible dentro de su propia clase, indistintamente del modificador de acceso que utilicemos. Es decir, desde dentro de la definición de una clase siempre tendremos acceso a todos sus atributos y podremos invocar cualquiera de sus métodos.

```
public class A { //clase pública
    int dato; //su ámbito es toda la clase:
    ... // el atributo dato es accesible desde cualquier lugar de A
}
```


Modificadores de acceso para miembros

Visibilidad por defecto

Cuando queremos acceder a miembros de otra clase hay diversos grados de visibilidad. La **visibilidad por defecto** es aquella que se aplica a miembros que declarados sin ningún modificador de acceso.

La visibilidad por defecto hace que un miembro sea visible desde las clases vecinas, pero invisible desde clases externas.

Modificadores de acceso para miembros

Modificador de acceso **private** y **public**

Con el modificador **private** obtenemos una visibilidad más restrictiva que por defecto, ya que impide el acceso incluso para las clases vecinas. Cualquier miembro declarado privado es invisible desde fuera de la clase.

El modificador **public** hace que un miembro sea visible incluso desde las clases externas previa importación. Otorga visibilidad total.

El modificador **private** se usa cuando queremos controlar los cambios de un atributo o cuando deseamos que no se conozca directamente su valor, o bien cuando queremos que un método solo sea invocado desde otros métodos de la clase, pero no fuera de ella. El acceso a esos miembros privados deberá hacerse a través de algún método **public** de la misma clase.

Modificadores de acceso para miembros

Modificador de acceso private y public

Ejemplo: clase Alumno con los atributos nombre y nota media. Esta última será un atributo privado, ya que interesa controlar el rango de valores válidos, que estarán comprendidos entre 0 y 10, inclusive. El método público asignarNota() será el encargado de controlar el valor asignado a la nota.

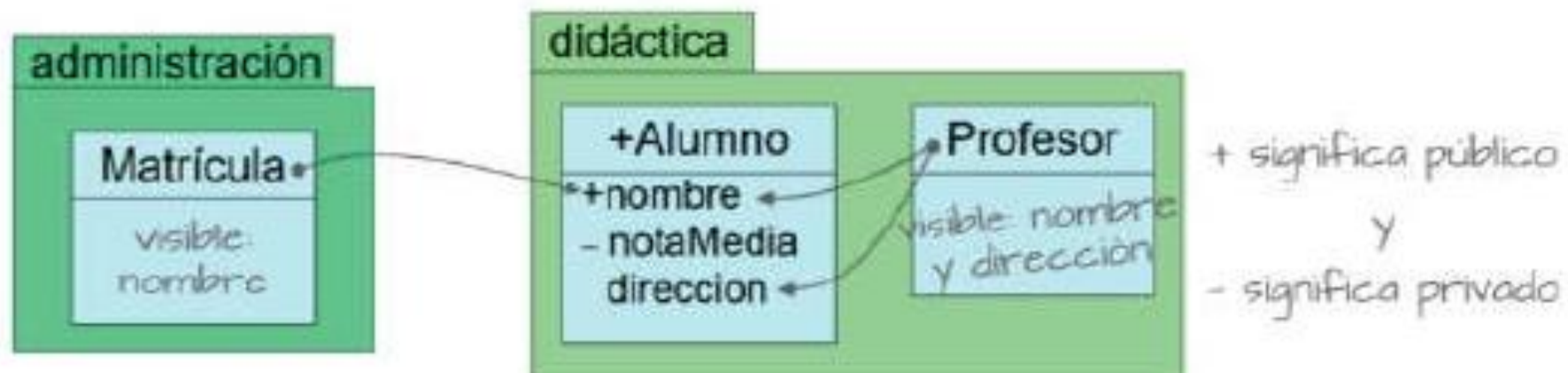
Solo será posible modificar la nota a través del método que la controla.

```
public class Alumno {  
    public String nombre; //atributo público  
    private double notaMedia; //atributo privado  
    String direccion; //atributo con visibilidad por defecto  
  
    public void asignaNota(double notaMedia) {  
        //nos aseguramos de que esté en el rango 0..10  
        if (notaMedia < 0 || notaMedia > 10) {  
            System.out.println("Nota incorrecta");  
        } else {  
            this.notaMedia = notaMedia;  
        }  
    }  
}
```

Modificadores de acceso para miembros

Modificador de acceso private y public

	Visible desde...		
	la propia clase	clases vecinas	clases externas
private	✓		
sin modificador	✓	✓	
public	✓	✓	✓



Encapsulación

La encapsulación en Java es un principio fundamental de la POO que consiste en agrupar los datos (atributos) y los métodos (comportamientos) relacionados con un objeto dentro de una sola unidad, llamada clase. Además, se controla el acceso a estos datos y métodos, permitiendo únicamente interacciones controladas desde el exterior de la clase.

La encapsulación permite **ocultar** la complejidad interna de una clase. Esto mejora la seguridad y la integridad de los datos, ya que solo se pueden modificar a través de métodos específicos de la clase, permitiendo así una validación adecuada antes de realizar cualquier cambio.

Encapsulación. Métodos get/set

Como hemos dicho, para evitar que un atributo public pueda ser modificado sin control previo, se aplicará la técnica de encapsulación, que consiste en ocultar atributos y, en su lugar, crear dos métodos públicos: **get** y **set**.

- El **método get**, devuelve el valor del atributo, lo que posibilita conocer su valor.
- El **método set**, permite asignar un valor al atributo, controlando el rango válido de valores.

Los métodos get/set (comúnmente llamados getters y setters) hacen que un atributo no visible se comporte como si lo fuera.

Encapsulación. Métodos get/set

Estos métodos se identifican con el get/set seguido del nombre del atributo.

Por ejemplo, para el atributo edad quedaría:

```
class Persona {  
    private int edad;  
    ...  
  
    public void setEdad(int edad) {  
        if (edad >= 0) //solo los valores positivos tienen sentido  
            this.edad = edad;  
        } // en caso contrario no se modifica la edad  
    }  
  
    public int getEdad() {  
        return edad;  
    }  
}
```

Encapsulación. Métodos get/set

Las ventajas de utilizar métodos get/set son que la implementación de la clase se encapsula, ocultando los detalles y, por otro lado, permite controlar qué atributos son accesibles para lectura y cuáles para escritura, así como los valores asignados.

Enumerados

Los **tipos enumerados** sirven para definir grupos de constantes como posibles valores de una variable. Por ejemplo, DiaDeLaSemana sería un tipo enumerado que puede tomar solo los valores constantes: LUNES, MARTES, ... DOMINGO. Su sintaxis sería:

```
enum DiaDeLaSemana{  
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO  
}
```

En la definición usamos la palabra clave **enum** y no **class**. En un programa se accede a sus valores de la forma DiaDeLaSemana.LUNES, DiaDeLaSemana.MARTES, etc.

Enumerados

Un tipo enumerado se puede implementar en un archivo aparte, como si fuera una clase, o bien dentro de la definición de la clase donde se va a usar.

Si queremos guardar en una variable el día de la semana, se haría:

```
DiaDeLaSemana día = DiaDeLaSemana.LUNES
```

Cuando tenemos que introducir por teclado un valor de tipo enumerado, escribiremos una cadena como “LUNES” y no DiaDeLaSemana.LUNES. Para asignarlo a una variable de tipo DiaDeLaSemana, tendremos que convertirla en el valor enumerado correspondiente. Para eso se usa el método **valueOf()** que convierte la cadena “LUNES” en el valor DiaDeLaSemana.LUNES.

Enumerados

Ejemplo:

```
Scanner sc = new Scanner(System.in);  
String día = sc.nextLine();    //introducimos LUNES  
DiaDeLaSemana díaSemana = DiaDeLaSemana.valueOf(día);
```

Enumerados

Si vamos a usar un tipo enumerado exclusivamente dentro de una clase, se puede definir dentro de ella. Por ejemplo, para añadir a la clase Cliente el atributo genero con los posibles valores MASCULINO y FEMENINO, definimos el tipo enumerado Genero dentro de la clase:

```
class Cliente{  
    enum Genero {MASCULINO, FEMENINO}  
    //definición del tipo enumerado  
    Genero genero;  
    //declaración de un atributo de tipo enumerado  
    . . .  
}
```

Enumerados

En el ejemplo anterior, el tipo Genero solo es accesible directamente desde dentro de la propia clase. Al escribir el constructor de Cliente, tenemos dos opciones para el parámetro de entrada del atributo genero:

1. Definirlo de tipo String y convertirlo en Genero dentro del código del constructor.

```
Cliente (... , String genero) {  
    . . .  
    this.genero=Genero.valueOf(genero);  
}
```

2. Definirlo de tipo Genero directamente.

```
Cliente (... , Genero genero) {  
    . . .  
    this.genero=genero;  
}
```

Enumerados

En el primer caso, cuando se llame al constructor, se le pasará una cadena.

```
String genero=new Scanner(System.in).next();  
Cliente c=new Cliente(..., genero);
```

En el segundo caso, habrá que hacer la conversión de String a Genero antes de llamar al constructor. Dicha conversión dependerá de dónde está definido el tipo enumerado.

a) Si está definido dentro de la clase Cliente, se accede a él con el nombre de la clase:

```
Cliente c=new Cliente(..., Cliente.Genero.valueOf(genero));
```

b) Si se ha definido en un archivo propio, aunque dentro del mismo paquete:

```
Cliente c=new Cliente(..., Genero.valueOf(genero));
```

Los tipos enumerador se pueden definir en paquetes distintos a donde se vayan a usar. En este caso, habrá que definirlos public e importarlos igual que si fueran clases.